

声指令随 - EpochPoint

北京邮电大学智能机器人实验期末大作业

任志诚 2023212020 梁恒 2023212008 金泽林 2023212012

BUPT

May 9, 2026

Content

任务设计

本模块设计完整多模态交互任务，执行流程如下：

- (1). 任务空间约束：机器人与测试者保持共线部署，机器人初始朝向任意。
- (2). 语音唤醒与声源粗对准：机器人接收唤醒词（如“鲁班鲁班”），通过声源定位实现朝向对准。
- (3). 视觉目标检测与趋近：识别举手姿态目标，执行离散视觉伺服，移动至目标前方预设安全距离。
- (4). 360° 全景视觉搜索：机器人摄像头进行全域旋转扫描，精准识别手指指向向量。
- (5). 视觉伺服与视场对准：微调摄像头姿态，将手指指向目标纳入相机视场。
- (6). 图像采集与云端推理：截取视觉帧并上传至 Docker 容器化部署的智能体，结合提示词完成云端多模态大模型推理。
- (7). 语音播报输出：接收云端分析结果，驱动语音合成节点完成内容播报。

初始需求分析

初始需求为“机器人听到呼唤后，根据声源定位并移动至人类身旁”。在实际的硬件评估中，存在以下致命物理约束：

- (1). 双足步态引发的视觉撕裂：目标设备为人形双足机器人，底盘在行走时身体会产生剧烈的钟摆式晃动，导致视觉帧产生毁灭性的运动模糊，实时动态视觉伺服在物理上不可行；
- (2). 多模态对齐灾难：在嘈杂的人群中，单凭声学无法精准锁定发声者的物理坐标

架构重构

鉴于此，为确保演示验收的 100% 成功率与系统的绝对安全，本模块做出两项核心工程妥协：

- (1). 摒弃“实时边走边看”，采用**走停看 (Stop-and-Go)** 异步离散状态机架构。在时间轴上严格物理隔离“视觉感知”与“步态移动”
- (2). 摒弃严格的声源定位，采用**声源初定位 + 举手姿态检测**的寻找方式。将复杂的声学测向转化为高鲁棒性的视觉目标锁定

ROS 订阅端 (异步非阻塞)

- 听觉触发源: `/micarrays/wakeup`
 - 被动监听唤醒信号, 触发状态流转
- 物理安全锁: `/MediumSize/BodyHub/WalkingStatus`
 - 高频缓存底盘状态, 是开启视觉调用的安全红线
- 原始视觉缓冲: `/chin_camera/image`
 - 维护长度为 1 的图像队列, 保证最新高清帧可用

ROS 服务与发布端

- 视觉服务端 (阻塞请求)
 - 目标锁定引擎: `/ros_gesture_node/gesture_detect`
 - 传入图像帧, 返回手势人体边界框坐标与尺寸
- 运动控制端 (离散指令)
 - 视野索敌: `/MediumSize/BodyHub/HeadPosition`
 - 离散步态: `/gaitCommand`

有限状态机 (FSM) 总览

核心四状态流水线：

1. STATE_IDLE：待机监视
2. STATE_VISION_LOCK：静止视觉锁定
3. STATE_BLIND_WALK：离散盲走
4. STATE_WAIT_FOR_STOP：等待步态停止

状态 0：待机监视 (STATE_IDLE)

- 执行逻辑：底盘锁定，头部居中，静默更新最新图像帧
- 流转条件：唤醒标志置为 True → 进入视觉锁定状态

状态 1：视觉锁定 (STATE_VISION_LOCK)

- 前置检查：必须静止，行走中则跳过当前帧
- 检测逻辑：调用手势检测，未找到则头部偏转，累计 3 次失败返回待机
- 目标捕获：提取边界框，计算像素偏置 Δx 与面积占比 $Area_Ratio$
- 流转： $Area_Ratio > 35\%$ $\rightarrow$ 到达；否则进入盲走

状态 2 与 3：盲走与等待停止

- STATE_BLIND_WALK (离散盲走)
 - 下发步数与转向角指令，立即切换至等待停止
- STATE_WAIT_FOR_STOP (阻塞监视)
 - 清空视觉缓存，仅轮询底盘状态
 - 恢复静止 → 切回视觉锁定

关键异常与防御机制

1. 指令冲突防御：非 IDLE 状态无视新唤醒信号
2. 底层断联看门狗：底盘状态超时 1.0 秒强制刹车
3. 图像过期防御：帧延迟超 0.5 秒拒绝识别，防止撞墙

工程背景与物理约束

初始需求：机器人到达用户附近后，依据用户手指指向，将头部转向目标物品并截图。实际评估存在核心约束：

- (1). 2D 视觉空间降维灾难：单目 RGB 丢失深度信息，纯视觉深度估计误差大，指向精度不可控
- (2). 脸部参照缺失与姿态抖动：近距离 FOV 受限，无眼部/肩部参照；手指生理性微颤导致远端射线放大偏移

架构重构决策

为保证空间定位精度与动作平滑性，采用两项核心方案：

- (1). 摒弃 2D 估计，采用 **3D 点云强相交**：使用 RealSense RGB-D 获取对齐彩色图与稠密点云，转化为严格 3D 射线与空间求交计算
- (2). 摒弃视线参照，采用**食指局部自对齐**：使用食指 MCP-TIP 向量，结合**卡尔曼滤波**平滑关键点，抑制高频抖动

ROS 系统接口映射规范

主控节点 `module2_attention_node` 作为承上启下中枢，接管模块一控制权，为后续模块提供高质量目标切片。

ROS 订阅端（异步非阻塞）

- RGB 图像流: `/camera/color/image_raw`
 - 用于 MediaPipe 骨架提取与最终截图存档
- 物理三维点云: `/camera/depth/color/points`
 - 维护 3D 环境模型, 为射线求交提供空间依据
- 模块一握手信号: `/module1/interaction_ready`
 - 接收到达并停稳信号, 激活本模块视觉管线

ROS 服务端（阻塞控制）

- 头部云台控制服务：

MediumSize/HeadController/HeadJointTrajectory

- 传入 Yaw、Pitch 角度，阻塞调用确保头部到位停稳，避免运动模糊

有限状态机 FSM 总览

本模块采用四状态流水线：

1. STATE_STANDBY：静默待命
2. STATE_RAY_CASTING：骨架提取与空间射线投射
3. STATE_TF_SOLVING：坐标变换与逆运动学解算
4. STATE_EXECUTE_CAPTURE：运动闭环与截帧捕获

状态 0：静默待命 (STATE_STANDBY)

- 执行逻辑：视觉管线休眠，降低算力消耗，监听模块一就绪信号
- 流转条件：收到 `interaction_ready = True` → 进入射线投射状态

状态 1：射线投射 (STATE_RAY_CASTING)

- 前置检查：RGB 与点云时间戳错位 $\leq 100\text{ms}$ ，否则丢弃
- 执行流程：
 - 提取食指 MCP 与 TIP 的 2D 坐标
 - 映射得到相机坐标系下 3D 坐标
 - 滤波后生成 3D 指向向量 $\vec{V}_{dir} = P_{tip} - P_{mcp}$
 - 射线求交，获取目标点云质心 P_{target}
- 流转：解算有效 $\rightarrow$ 进入坐标变换；超时则语音提示

状态 2：坐标变换与 IK 解算 (STATE_TF_SOLVING)

- 执行逻辑：
 1. 通过 ROS tf 将目标点从相机坐标系转换至头部云台基座坐标系
 2. 逆运动学求解云台角度：

$$\text{Yaw} = \arctan 2(Y_t, X_t)$$

$$\text{Pitch} = \arctan 2\left(Z_t, \sqrt{X_t^2 + Y_t^2}\right)$$

- 流转：解算完成 → 进入执行捕获状态

状态 3：执行与截图 (STATE_EXECUTE_CAPTURE)

- 下发 Yaw、Pitch 至头部控制器，阻塞等待到位
- 延时 0.5s 消除机械震颤，清空缓存并截取静止清晰帧
- 保存高清截图，发布任务完成标志
- 流转：截图完成 → 回到待命状态

关键异常与边界防御机制

1. 射界逃逸防御：指向天空/超 3m 无效目标，拒绝 IK 解算
2. 关节物理限位：Pitch 限制在 $-30^{\circ} \sim 45^{\circ}$ ，超限截断保护舵机
3. 视觉防抖屏蔽：截图前清空缓存，仅取静止后首帧干净画面

Langchain 是什么？

LangChain 是一个用于构建大语言模型 (LLM) 应用的主流开源框架，核心是连接模型、数据与工具，解决 LLM 孤立、无记忆、难执行复杂任务的问题。总而言之，可以用起开发 agent 从而实现 (在我们的设计中) 如下功能



使用 LangChain 的必要性

调研机器人配置，得到如下信息：

- (1). CPU：4 核 8 线程 (i5-8259U)，无独立显卡，本地推理大模型能力受限。
- (2). 内存与存储：仅 8GB 内存，116GB 系统盘，难以加载或运行 7B 以上量级的 LLM。
- (3). 系统环境：Ubuntu 16.04 (较老)，Python 2.7/3.5 为主，不直接支持多数新 LLM 框架。
- (4). 开机后负载极低 (CPU 空闲 > 0.97)，主要运行 MQTT 消息服务器 (EMQX)，适合作为调用云端 LLM 的轻量 Agent。

可见，机器人的配置十分低端，根本无法进行本地图像识别；为了配置本地和云端 LLM 之间的通信，利用 LangChain 搭建一个简单的 agent 是一个可行的方案

初步的方案 1

由于机器人全局环境为 python3.8，而 LangChain 推荐的 python 环境为 3.10+，因此这里通过 Docker 构建了一个独立的运行环境：

- (1). 基础系统为 Ubuntu 20.04，用于隔离机器人原有的老旧系统环境。
- (2). 在容器内手动编译并安装 Python 3.10，满足 LangChain 对较新 Python 版本的要求。
- (3). 安装 ROS Noetic 相关组件，保留与机器人现有消息通信和中间件对接的能力。
- (4). 安装 LangChain、langchain-community 和 langchain-openai，作为本地 Host 调用云端 LLM 的核心依赖。

因此，该 Docker 环境本质上是一个“ROS 通信 + Python 3.10 + LangChain”的轻量 Agent 运行容器，用于承载机器人端与云端大模型之间的交互逻辑。

初步的方案 2

很容易想到分三步实现，第一步为与全局环境通信，需要弄清楚如何将机器人截屏的图像传输到 Docker 环境中，第二步为将接受的到的图像上传到云端 LLM，第三步为将接收到的文本内容，通过调用相关节点，读出来。可见实现的难点主要在通信上

通信方案设计

ROS 系统的两种通信方式 Topic 和 Service 均要求通信双方为 Node 类型；Node 在 Package 下，故第一步创建了一个 Package：

```
catkin_create_pkg ros_langchain_node roscpp rospy  
std_msgs geometry_msgs
```

然后生成如下目录结构：

```
ros_langchain_node/  
├── CMakeLists.txt  
├── include/  
│   └── ros_langchain_node/  
├── package.xml  
└── src/
```